

Projektdokumentation

Projektthema

Entwicklung einer konsolenbasierten Bestands- und Bewegungsverwaltung für Lagerartikel mit SQLite-Datenhaltung, klarer Schichtentrennung und automatisierten Tests.

1. Einleitung

In diesem Projekt wird eine konsolenbasierte Bestands- und Bewegungsverwaltung für Lagerartikel entwickelt, die eine SQLite-Datenbank als persistente Datenhaltung nutzt und durch eine klare Schichtenarchitektur sowie automatisierte Tests abgesichert wird. Ziel ist es, eine wartbare, erweiterbare und fachlich saubere Anwendung zu erstellen, die die typischen Aufgaben einer Lagerverwaltung in einem reduzierten, aber praxisnahen Umfang abbildet.

2. Ausgangssituation und Problemstellung

2.1 Ausgangssituation

Im betrieblichen Umfeld von SSI Schäfer IT Solutions spielt die strukturierte Verwaltung von Lager- und Bewegungsdaten eine wichtige Rolle, da logistische Prozesse von korrekten Beständen, nachvollziehbaren Buchungen und einer konsistenten Datenhaltung abhängen. Das Projekt wurde als Ausbildungs- und Entwicklungsprojekt in vereinfachter Form umgesetzt, orientiert sich jedoch fachlich an typischen Anforderungen aus der Lager- und Bestandsverwaltung.

Zu Beginn lag bereits eine erste Konsolenanwendung vor, mit der grundlegende Funktionen wie das Anlegen von Artikeln, die Anzeige aller Artikel, die Suche nach Artikelnummer oder Name sowie einfache Bestandsänderungen umgesetzt werden konnten. Diese frühe Version arbeitete zunächst speicherbasiert und später dateibasiert, war jedoch architektonisch noch nicht konsequent in Schichten getrennt und daher nur eingeschränkt wartbar und erweiterbar.

Im weiteren Verlauf wurden bereits mehrere funktionale Erweiterungen umgesetzt, darunter eine Lagerbewegungshistorie, Dateimport und -export, Refactoring zur verbesserten Eingabeverarbeitung, statistische Auswertungen sowie automatisierte Tests mit JUnit 5 und Maven. Zusätzlich wurde mit SQLite und JDBC ein erster Datenbankzugriff geschaffen. Damit bestand eine gute technische Basis, jedoch fehlte noch die saubere Zusammenführung zu einer klar strukturierten, dokumentierbaren Gesamtanwendung.

2.2 Problemstellung

Die vorhandene Anwendung deckte wesentliche fachliche Grundfunktionen bereits ab, jedoch waren einige projektentscheidende Aspekte noch nicht ausreichend umgesetzt. Insbesondere fehlte eine klare Trennung zwischen Benutzerschnittstelle, Anwendungslogik und Persistenz, wodurch Änderungen am Programmaufbau mit erhöhtem Wartungsaufwand verbunden gewesen wären.

Zudem war die Datenhaltung zunächst überwiegend dateibasiert organisiert. Für einfache, erste Lernphasen ist das ausreichend, bei wachsender Komplexität entstehen jedoch Nachteile hinsichtlich Struktur, Erweiterbarkeit und Nachvollziehbarkeit. Für eine realitätsnähere und fachlich sauberere Lösung sollte daher eine relationale SQLite-Datenhaltung eingebunden werden, die über eine klar definierte Persistenzschicht angesprochen wird.

Außerdem sollte das Projekt nicht nur funktional arbeiten, sondern auch die für die Abschlussprüfung relevanten Qualitätsmerkmale erfüllen. Dazu zählen nachvollziehbare Anforderungen, eine begründete Architekturentscheidung, eine strukturierte Projektplanung, automatisierte Tests, eine wirtschaftliche Betrachtung und eine saubere Dokumentation der Umsetzung.

3. Zielsetzung

3.1 Projektziel

Ziel des Projekts war die Entwicklung bzw. Weiterentwicklung einer konsolenbasierten Anwendung zur Verwaltung von Lagerartikeln und Lagerbewegungen. Die Anwendung sollte es ermöglichen, Artikel anzulegen, Bestände kontrolliert zu erhöhen oder zu verringern, Mindestbestände zu prüfen, Buchungshistorien nachzuvollziehen und ausgewählte Auswertungen zur Lagersituation bereitzustellen.

3.2 Qualitative Ziele

Ein wesentliches Teilziel bestand darin, die Anwendung architektonisch so zu überarbeiten, dass eine klare Schichtentrennung entsteht. Die Konsolenoberfläche sollte ausschließlich für Ein- und Ausgaben zuständig sein, die fachliche Verarbeitung über eine Service-Schicht erfolgen und die Datenbankzugriffe über eine eigene Persistenzschicht gekapselt werden. Diese Trennung verbessert Wartbarkeit, Testbarkeit und Erweiterbarkeit der Anwendung.

Ein weiteres Ziel war die Einführung bzw. Konsolidierung einer SQLite-basierten Datenhaltung für die spezialisierten Datensätze rund um Pokémon-Karten beziehungsweise lagerähnliche Artikelstrukturen. Die Anbindung sollte mittels JDBC erfolgen und über Repository- bzw. Datenbankklassen organisiert werden.

Zusätzlich sollte die Softwarequalität durch automatisierte Unit-Tests abgesichert werden. Kritische Fachlogik wie Bestandsänderungen, Validierungen und Statistikfunktionen sollte mit JUnit 5 getestet werden, um Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Anwendung zu erhöhen.

3.3 Nichtziele (Abgrenzung)

Nicht Ziel des Projekts ist die Entwicklung einer grafischen Benutzeroberfläche oder einer Weboberfläche. Die Bedienung erfolgt vollständig über die Konsole. Auch eine umfassende Benutzer- und Rechteverwaltung sowie ein produktiver Einsatz in einem realen Lagerbetrieb sind nicht Bestandteil dieses Projektes, sondern werden lediglich als mögliche Erweiterungen für die Zukunft betrachtet.

4. Projektumfeld

Das Projekt wurde im Ausbildungsumfeld bei SSI Schäfer IT Solutions GmbH, Standort Giebelstadt, durchgeführt. SSI Schäfer ist im Bereich Intralogistik und Softwarelösungen für Lager- und Materialflussprozesse tätig, wodurch das Projektthema fachlich gut in das Unternehmensumfeld passt.

Die Entwicklung erfolgte mit Java in einer Maven-basierten Projektstruktur. Für die Tests wurde JUnit 5 verwendet, und die Datenbankbindung wurde mit SQLite über JDBC umgesetzt. Als Entwicklungsumgebung kamen typische Werkzeuge der Java-Entwicklung zum Einsatz, wodurch das Projekt auch technisch dem Ausbildungsstand eines Fachinformatikers für Anwendungsentwicklung entspricht.

5. Anforderungen

5.1 Funktionale Anforderungen

Die Anwendung musste mindestens folgende funktionalen Anforderungen erfüllen:

- Anlegen und Verwalten von Artikeln.
- Anzeigen aller vorhandenen Artikel in strukturierter Form.
- Suche nach Artikeln über Artikelnummer oder Bezeichnung.
- Bestandsänderungen über Zugang und Abgang mit fachlicher Prüfung, dass kein negativer Lagerbestand entsteht.
- Prüfung von Mindestbeständen und Ausgabe kritischer Artikel.
- Verwaltung und Anzeige einer Bewegungshistorie mit Zeitstempel.

- Bereitstellung einfacher Lagerstatistiken wie Artikelanzahl, Gesamtbestand und Anzahl kritischer Bestände.
- Speicherung und Laden ausgewählter Datenbestände über Datei- oder Datenbankmechanismen.

Das Use-Case-Diagramm zeigt die fachlichen Anwendungsfälle der Lagerverwaltungsanwendung aus Sicht des Benutzers. Der Benutzer kann Artikel anlegen, anzeigen und suchen sowie Lagerzugänge und Lagerabgänge durchführen. Darüber hinaus ermöglicht die Anwendung die Prüfung kritischer Mindestbestände, die Anzeige der Bewegungs-Historie und die Bereitstellung einfacher Lagerstatistiken. Ergänzend wurden Funktionen für das Speichern und Laden von Pokémon-Karten über eine SQLite-Datenbank berücksichtigt. Das Diagramm macht deutlich, dass die Anwendung auf typische Aufgaben der Bestands- und Bewegungsverwaltung ausgerichtet ist.

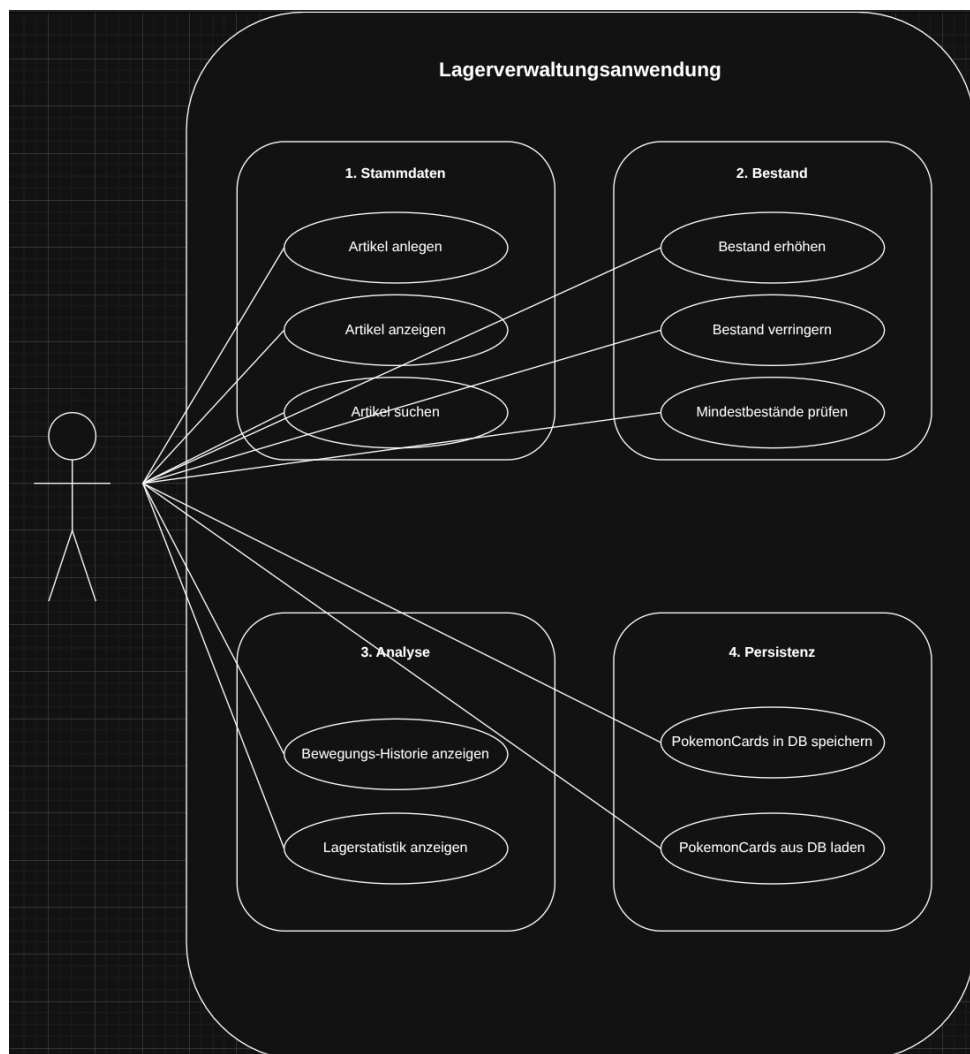


Abbildung 1: Use-Case-Diagramm der Lagerverwaltungsanwendung

5.2 Nichtfunktionale Anforderungen

Neben den fachlichen Anforderungen wurden auch nichtfunktionale Anforderungen definiert:

- Die Anwendung sollte modular und wartbar aufgebaut sein.
- Die Architektur sollte eine klare Trennung zwischen UI, Service, Domäne und Persistenz aufweisen.
- Fehlerhafte Eingaben sollten abgefangen und verständlich behandelt werden.
- Die Fachlogik sollte automatisiert testbar sein.
- Die Datenhaltung sollte lokal ohne zusätzlichen Serveraufwand funktionieren, weshalb SQLite als geeignete Lösung gewählt wurde.
- Die Anwendung sollte für ein Ausbildungsprojekt mit vertretbarem Implementierungsaufwand realisierbar bleiben.

5.3 Abgrenzung

Nicht Bestandteil des Projekts war die Entwicklung einer grafischen Benutzeroberfläche oder einer Weboberfläche. Die Bedienung erfolgt vollständig über die Konsole. Auch eine umfassende Benutzer- und Rechteverwaltung wurde nur als mögliche spätere Erweiterung betrachtet und nicht vollständig umgesetzt.

Ebenso war die Migration zu einer vollständigen Spring-Boot-REST-Anwendung nicht Bestandteil des aktuellen Projekts, sondern wurde als möglicher nächster Entwicklungsschritt bewertet. Damit blieb der Projektumfang auf eine fachlich sinnvolle, aber für die IHK realistisch beherrschbare Lösung begrenzt.

5.4 Lösungsansatz

Zur Erreichung der Projektziele wurde die Anwendung in eine mehrschichtige Struktur überführt. Die Konsolenoberfläche übernimmt ausschließlich die Interaktion mit dem Benutzer, etwa Menüführung, Eingabeaufforderungen und Ausgaben. Die Geschäftslogik, etwa das Anlegen von Artikeln, das Buchen von Bestandsänderungen oder die Ermittlung von Lagerstatistiken, wird in den fachlichen Klassen gekapselt. Die fachlichen Regeln verbleiben in den Domänenklassen wie Inventory, Article, BaseArticle, PokemonCard und StockMovement.

Für die Persistenz wurde ein Repository-Ansatz gewählt. Datenbankzugriffe erfolgen nicht direkt aus der UI oder aus der Fachlogik, sondern ausschließlich über Klassen wie PokemonCardRepository und DatabaseManager. Dadurch wird die

Anwendung besser wartbar, da Änderungen an der Datenhaltung die übrigen Teile nur minimal beeinflussen.

Die Entscheidung für SQLite fiel, weil die Lösung für ein lokales Ausbildungsprojekt ohne separaten Datenbankserver auskommt und sich über JDBC direkt in eine Java-Anwendung integrieren lässt. Damit konnte eine relationale Datenhaltung umgesetzt werden, ohne zusätzliche Infrastruktur bereitstellen zu müssen.

5.5 Wirtschaftliche Betrachtung

Die Entwicklung einer eigenen konsolenbasierten Bestands- und Bewegungsverwaltung ist wirtschaftlich vor allem dann sinnvoll, wenn damit gezielt Ausbildungsziele, Architekturprinzipien und fachliche Zusammenhänge praxisnah vermittelt werden. Für den produktiven Einsatz in größerem Maßstab würden in der Regel bestehende, professionell integrierte Lagerverwaltungslösungen bevorzugt; als Ausbildungs- und Prüfungsprojekt ist die Eigenentwicklung jedoch sinnvoll, da sie die vollständige Bearbeitung eines Softwarelebenszyklus ermöglicht.

Durch die Wahl von Java, Maven, JUnit 5 und SQLite wurden etablierte und im Ausbildungskontext gut verfügbare Technologien eingesetzt. SQLite verursacht keinen separaten Serverbetrieb, was Einrichtungs- und Wartungsaufwand reduziert. Die modulare Architektur und automatisierten Tests senken zusätzlich die langfristigen Änderungs- und Fehlersuchkosten, da Erweiterungen und Refactorings kontrollierter durchgeführt werden können.

6. Projektplanung

6.1 Arbeitspakete

Für eine Projektplanung sollte das Vorhaben in klar abgegrenzte Arbeitspakete zerlegt werden. Typische Pakete für dieses Projekt sind Analyse der Ausgangssituation, Anforderungsdefinition, Architektur- und Datenbankentwurf, Refactoring der Schichtenstruktur, Implementierung der SQLite-Persistenz, Testkonzeption und Testdurchführung, Dokumentation sowie Projektabschluss. Es wird eine nachvollziehbare Zeit-, Ressourcen- und Kostenplanung dargestellt. Außerdem wird bei der Durchführung ein Vergleich von Soll- und Ist-Zeiten aufgezeigt.

6.2 Zeitplanung

80-Stunden-Struktur:

- Analyse und Zieldefinition: 8 Stunden.
- Fachliches und technisches Konzept: 10 Stunden.
- Refactoring und Schichtentrennung: 16 Stunden.
- SQLite-Anbindung und Repository-Umsetzung: 14 Stunden.
- Implementierung und Integration der Funktionen: 14 Stunden.
- Testkonzept, Unit-Tests und Testdurchführung: 8 Stunden.
- Dokumentation, Übergabe und Reflexion: 10 Stunden.

6.3 Ressourcenplanung

Für die Durchführung des Projekts werden folgende Ressourcen benötigt:

- Entwicklungsrechner mit Java-Entwicklungsumgebung (z. B. IntelliJ IDEA).
- Java Development Kit (JDK).
- Maven als Build-Werkzeug.
- JUnit 5 für automatisierte Tests.
- SQLite Datenbank und JDBC-Treiber.
- Versionsverwaltung (z. B. Git).

7. Lösungsentwurf

7.1 Architektur

Die Zielarchitektur der Anwendung basiert auf einer klaren Schichtenstruktur. Die Benutzerschnittstelle ist für die Menüführung sowie für Ein- und Ausgaben zuständig. Die fachliche Verarbeitung erfolgt in der Domänenschicht, während der Datenbankzugriff vollständig über die Persistenzschicht gekapselt wird. Dadurch werden direkte Abhängigkeiten zwischen Benutzeroberfläche und Datenbank vermieden, was die Wartbarkeit, Testbarkeit und Erweiterbarkeit der Anwendung verbessert.

Die Benutzerschnittstelle umfasst die Main-Klasse sowie das ConsoleMenu. Diese Komponenten übernehmen ausschließlich die Interaktion mit dem Benutzer und enthalten keine eigene fachliche Verarbeitung. Die Domänenschicht bildet das fachliche Zentrum der Anwendung. Insbesondere die Klasse Inventory koordiniert die Verwaltung von Artikeln, Beständen und Lagerbewegungen. Ergänzt wird diese Schicht durch die fachlichen Klassen BaseArticle, Article, PokemonCard und StockMovement.

Die Persistenzschicht kapselt sämtliche Zugriffe auf die SQLite-Datenbank. Hierzu dienen insbesondere die Klassen PokemonCardRepository und DatabaseManager. Durch diese Trennung bleibt die übrige Anwendung unabhängig von konkreten Zugriffstechnologien und kann später leichter erweitert werden, beispielsweise um zusätzliche Persistenzmechanismen oder eine alternative Benutzerschnittstelle.

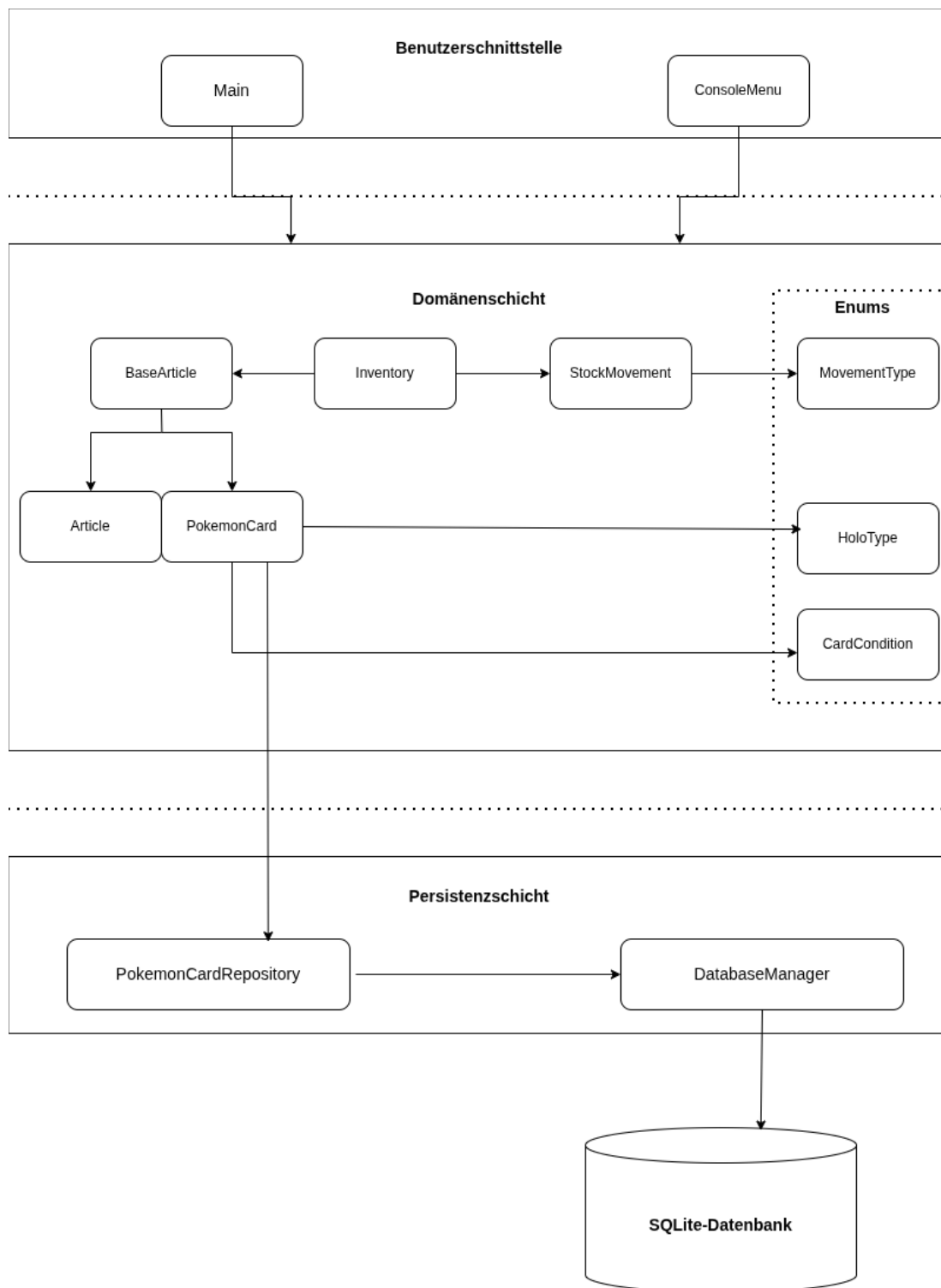


Abbildung 2: Architekturdiagramm der Schichtenarchitektur

Das Architekturdiagramm verdeutlicht die Aufteilung der Anwendung in Benutzerschnittstelle, Domänenschicht und Persistenzschicht. Es zeigt, dass die Daten von der Konsolenoberfläche über die fachliche Verarbeitung bis zur Datenbank in klar getrennten Verantwortungsbereichen verarbeitet werden. Diese Struktur unterstützt eine saubere Implementierung der fachlichen Logik und erleichtert spätere Erweiterungen.

Das UML-Klassendiagramm ergänzt diese architektonische Sicht durch die Darstellung der wichtigsten Klassen und ihrer Beziehungen. Es zeigt die Vererbung zwischen BaseArticle, Article und PokemonCard, die Aggregation von Artikeln und Lagerbewegungen in Inventory sowie die Anbindung der Persistenzklassen an die Datenbankzugriffe.

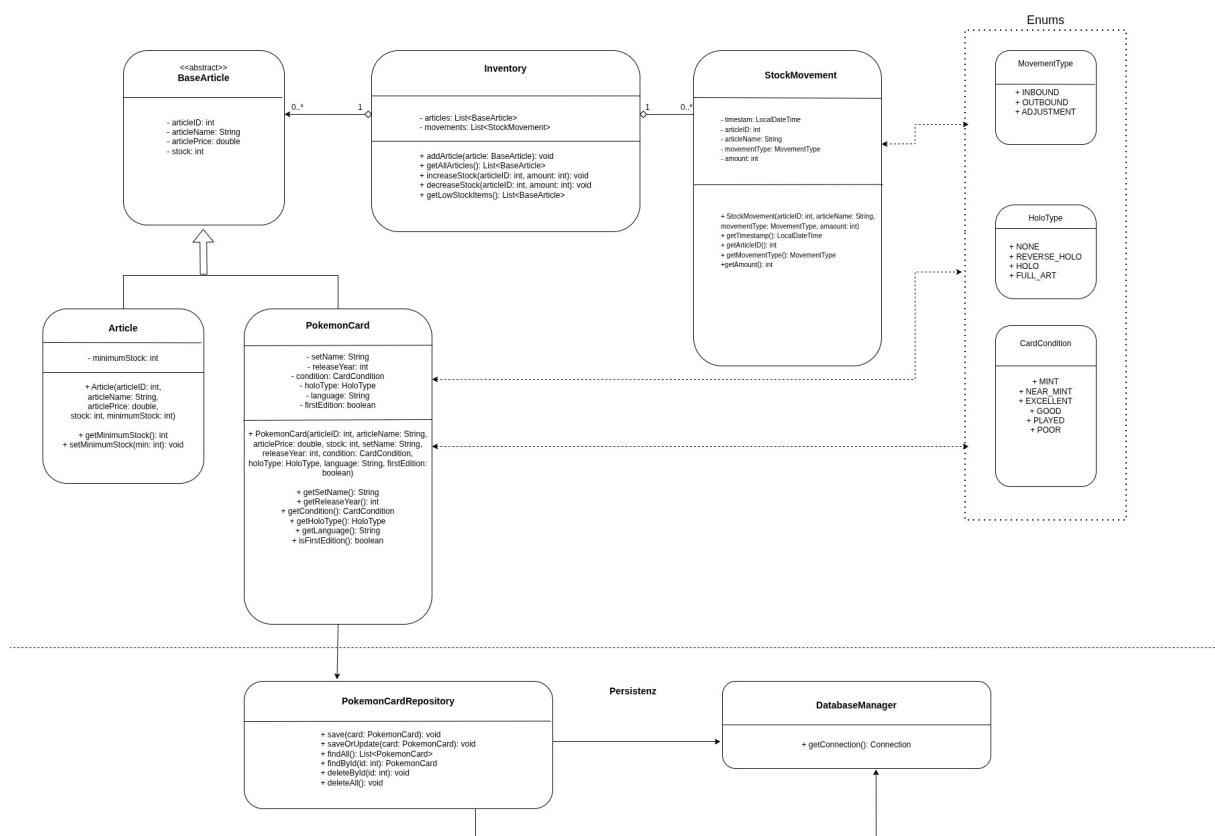


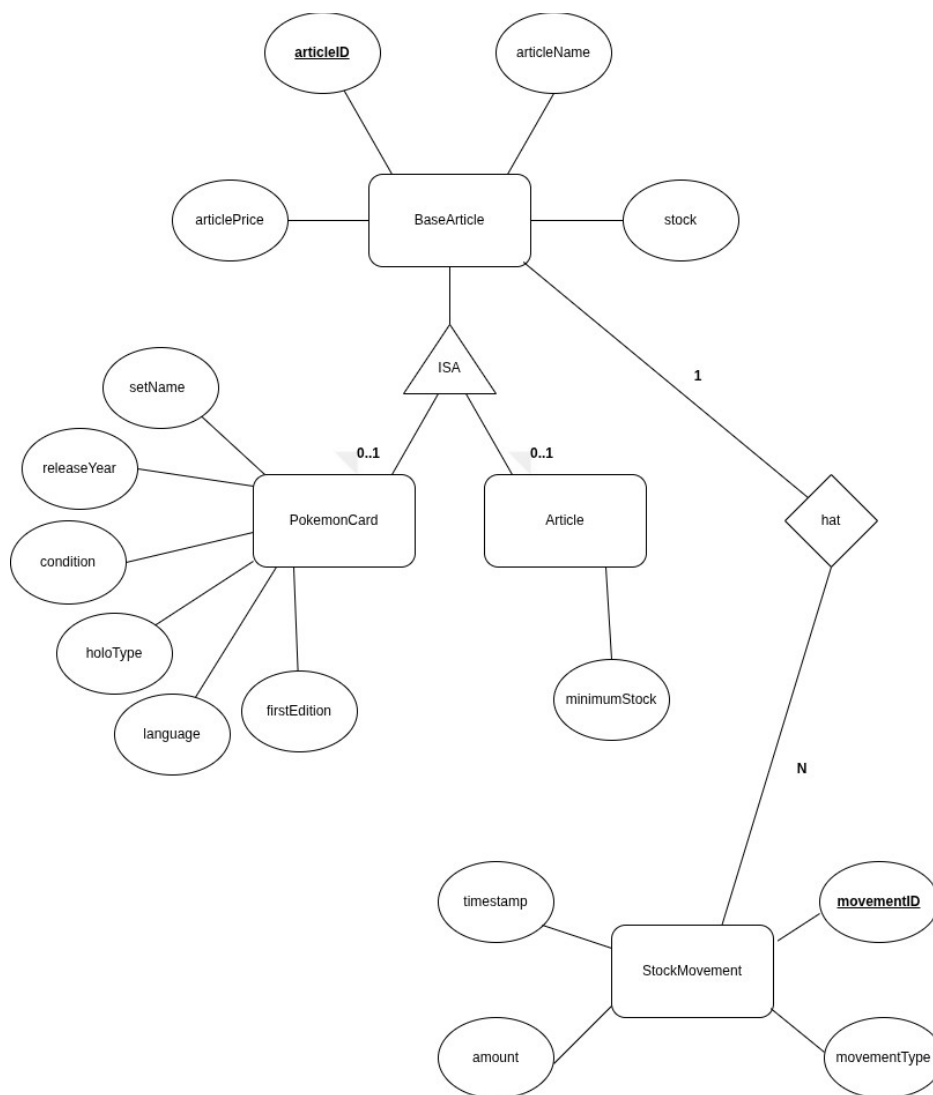
Abbildung 3: UML-Klassendiagramm der Kernstruktur

7.2 Datenmodell

Das Datenmodell beschreibt sowohl die fachliche Sicht auf die verwendeten Objekte als auch deren relationale Umsetzung in der Datenbank. Zunächst wird mit dem ER-Modell die fachliche Struktur der Entitäten und ihrer Beziehungen dargestellt. Darauf aufbauend zeigt das Tabellenmodell die konkrete Persistenzstruktur in SQLite.

Im ER-Modell dient BaseArticle als gemeinsame Basisentität für unterschiedliche Artikelarten. Die Entitäten Article und PokemonCard werden als Spezialisierungen dieses Basistyps modelliert. Zusätzlich bildet die Entität StockMovement die Lagerbewegungen ab, die zur Nachvollziehbarkeit von Zu- und Abgängen dienen. Zwischen einem Artikel und dessen Lagerbewegungen besteht eine 1:N-Beziehung, da ein Artikel mehrere Bewegungen besitzen kann, jede einzelne Bewegung jedoch genau einem Artikel zugeordnet ist.

Abbildung 4: ER-Modell der Lagerverwaltung



Das ER-Modell stellt damit die fachliche Grundlage des Datenmodells dar. Es zeigt, welche Informationen fachlich benötigt werden und in welcher Beziehung diese zueinander stehen. Besonders wichtig ist hierbei die Trennung zwischen den allgemeinen Artikelstammdaten und den spezialisierten Eigenschaften einer Pokémon-Karte.

Das Tabellenmodell zeigt anschließend die relationale Umsetzung der tatsächlich persistent gespeicherten Daten. Im Mittelpunkt steht die Tabelle `pokemon_cards`, in der die relevanten Eigenschaften einer Pokémon-Karte in relationaler Form abgelegt werden. Dazu zählen die Artikelnummer, der Name, der Preis, der Bestand, der Setname, das Erscheinungsjahr, der Kartenzustand, der Holo-Typ, die Sprache und die Information, ob es sich um eine First Edition handelt.

Als Primärschlüssel dient das Attribut `article_id`, da es jeden Datensatz eindeutig identifiziert. Das Tabellenmodell macht damit sichtbar, wie die fachlichen Informationen aus dem objektorientierten Modell in eine relationale Datenstruktur überführt werden.

pokemon_cards			
Spaltenname	Datentyp	Schlüssel	Beschreibung
article_id	STRING	PRIMÄRSCHLÜSSEL (PK)	Eindeutige Kennung des Artikels (Karte)
article_name	STRING		Name der Karte
article_price	DECIMAL		Preis der Karte
stock	STRING		Aktueller Lagerbestand
set_name	STRING		Name des Sets, zu dem die Karte gehört
release_year	INT		Erscheinungsjahr des Sets
card_condition	STRING		Zustand der Karte
holo_type	STRING		Holografischer Type der Karte
language	STRING		Sprache der Karte
first_edition	BOOLEAN		Gibt an, ob die Karte eine Erstauflage ist

Abbildung 5: Tabellenmodell der SQLite-Persistenz

8. Projektdurchführung

8.1 Analyse und Konzept

Im ersten Schritt wurde der bisherige Entwicklungsstand analysiert. Dabei wurden bereits vorhandene Funktionen, Klassenstrukturen und Persistenzmechanismen bewertet. Besonders relevant war die Erkenntnis, dass zwar viele fachliche Funktionen bereits existierten, diese aber noch nicht durchgängig in einer projekttauglichen Zielarchitektur organisiert waren.

8.2 Refactoring und Schichtentrennung

Anschließend wurde eine Zielarchitektur festgelegt. Die Konsole sollte als UI-Schicht bestehen bleiben, da sie den Projektumfang begrenzt und gleichzeitig die Konzentration auf Fachlogik, Architektur und Datenhaltung ermöglicht. Darüber wurde die fachliche Koordination so organisiert, dass die relevanten Use Cases strukturiert verarbeitet werden.

Danach erfolgte die Überarbeitung der Domäne. Die Klassen Article, Inventory und StockMovement wurden fachlich geschärft, Validierungen in Methoden und Konstruktoren verankert und Regeln wie „Bestand darf nicht negativ werden“ innerhalb der Objekte selbst umgesetzt. Die Einführung von BaseArticle als abstrakter Oberklasse ermöglichte eine sauberere Vererbung und Polymorphie, da eine Spezialisierung wie PokemonCard nicht sinnvoll aus einer zu speziell geschnittenen Article-Klasse abgeleitet werden sollte.

8.3 Implementierung der Persistenzschicht

Im Anschluss wurde die Persistenzschicht konkretisiert. Für datenbankgestützte Objekte wurde eine SQLite-Tabelle definiert, die Spalten wie article_id, article_name, article_price, stock, set_name, release_year, card_condition, holo_type, language und first_edition enthält. Über den SQLite-JDBC-Treiber und eine Datenbankklasse wurde das Öffnen der Verbindung, das Initialisieren der Tabelle sowie das saubere Schließen der Verbindung umgesetzt.

Darauf aufbauend wurde ein Repository implementiert, das Methoden zum Speichern, Laden und perspektivisch auch Aktualisieren oder Löschen von Datensätzen bereitstellt. Damit wurde sichergestellt, dass Datenbanklogik nicht direkt in der Benutzerschnittstelle oder in den Domänenklassen liegt, sondern zentral und austauschbar gekapselt ist.

Parallel zur Implementierung wurde die Qualitätssicherung ausgebaut. Mit JUnit 5 wurden fachlich relevante Funktionen getestet, darunter gültige und ungültige Bestandszugänge, zulässige und unzulässige Abgänge, Statistikfunktionen sowie Anpassungen nach Refactorings und Vererbungsänderungen.

8.4 Implementierung der fachlichen Funktionen

Die fachlichen Funktionen zur Artikelverwaltung, Bestandsführung, Bewegungsprotokollierung und Statistik wurden in die neue Struktur überführt. Menüfunktionen wie „Artikel anlegen“, „Bestand erhöhen“, „Bestand verringern“, „Mindestbestand prüfen“, „Bewegungshistorie anzeigen“ und „Lagerstatistik anzeigen“ wurden so angepasst, dass sie die Services ansprechen, welche wiederum die Domänen- und Persistenzklassen nutzen.

Bereits vorhandene Funktionen wie das Speichern und Laden von Artikeln in Textdateien wurden in die überarbeitete Struktur integriert und dienen ergänzend zur Datenbank als Import- oder Sicherungsmechanismus. Die fachliche Verarbeitung bleibt dabei unabhängig von der konkreten Form der Datenhaltung.

Das Sequenzdiagramm zeigt den Ablauf beim Speichern einer neuen Pokémon-Karte. Zunächst erfasst der Benutzer die Daten über die Konsolenoberfläche. Anschließend werden die Eingaben an die fachliche Logik übergeben, dort verarbeitet und an das Repository weitergereicht. Das Repository führt den eigentlichen Datenbankzugriff aus und speichert den Datensatz in der SQLite-Datenbank. Danach wird die erfolgreiche Rückmeldung über die beteiligten Schichten bis zur Benutzeroberfläche zurückgegeben.

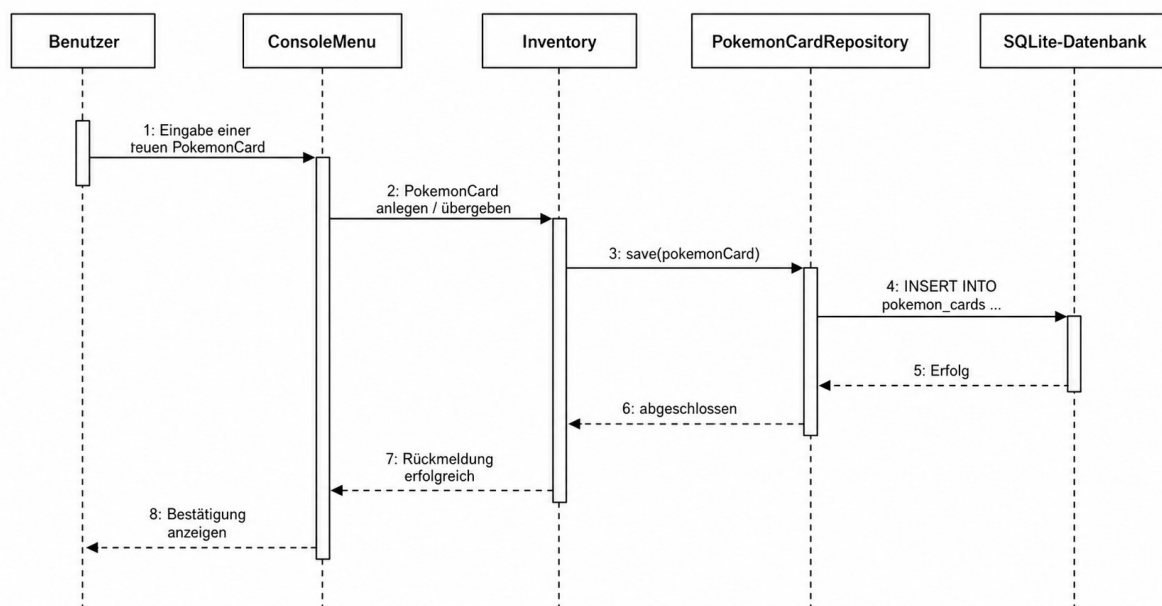


Abbildung 6: Sequenzdiagramm zum Speichern einer PokemonCard

Das Diagramm verdeutlicht damit den konkreten Lauf eines zentralen Anwendungsfalls über mehrere Schichten hinweg. Gleichzeitig wird sichtbar, dass die Benutzeroberfläche keine direkte Datenbanklogik enthält, sondern die Verarbeitung über klar definierte Verantwortlichkeiten erfolgt.

8.5 Testkonzept und Qualitätssicherung

Das Testkonzept orientierte sich an den zentralen fachlichen Risiken der Anwendung. Besonders kritisch waren Bestandsänderungen, da hier fehlerhafte Logik unmittelbar zu inkonsistenten Lagerständen führen würde. Deshalb wurden für die Methoden zur Bestandserhöhung und Bestandsverringerung sowohl positive als auch negative Testfälle definiert.

Zusätzlich wurden Tests für Statistikfunktionen, Objektvalidierungen und polymorphe Erweiterungen angelegt. Die automatisierten Tests reduzierten das Risiko, dass funktionierende Bestandteile durch spätere Umbauten unbemerkt beschädigt werden. Neben automatisierten Unit-Tests sind in der Projektdokumentation auch manuelle Funktionstests sinnvoll, etwa zur Menüführung, zur Konsolenausgabe, zum Laden aus Dateien oder zur Datenbankanzeige.

9. Testergebnisse

Die automatisierten Unit-Tests konnten erfolgreich ausgeführt werden und liefern grüne Testergebnisse für die zentralen fachlichen Funktionen. Dadurch wurde sichergestellt, dass Refactorings und Architekturänderungen die bestehenden Funktionen nicht unbeabsichtigt beschädigen.

Die manuellen Funktionstests zeigten, dass die Anwendung die definierten fachlichen Anforderungen erfüllt. Artikel können angelegt, gesucht und angezeigt werden, Bestände lassen sich nur in gültigen Grenzen verändern, Mindestbestände werden korrekt erkannt und Bewegungen nachvollziehbar protokolliert. Die Interaktion mit der SQLite-Datenbank funktioniert wie geplant, und die relevanten Datensätze können gespeichert und wieder geladen werden.

10. Projektergebnis

Als Ergebnis entstand eine lauffähige konsolenbasierte Anwendung zur Verwaltung von Lagerartikeln und Bewegungen, die über mehrere Entwicklungsstufen fachlich ausgebaut und architektonisch verbessert wurde. Das System unterstützt die Anlage und Verwaltung von Artikeln, Bestandsänderungen mit Validierung, Historienführung, einfache statistische Auswertungen sowie persistente Datenhaltung über Dateien und SQLite.

Besonders hervorzuheben ist die überarbeitete Struktur mit erkennbarer Trennung von UI, Fachlogik und Persistenz. Diese Schichtentrennung verbessert die Lesbarkeit des Codes, erleichtert spätere Erweiterungen und macht die Anwendung fachlich sauberer dokumentierbar. Zusätzlich wurde die Qualität durch automatisierte JUnit-

Tests abgesichert, wodurch eine solide Grundlage für Weiterentwicklungen geschaffen wurde.

11. Einführung und Nutzung

Da es sich um eine Konsolenanwendung handelt, ist die Einführung technisch vergleichsweise einfach. Nach dem Build mit Maven kann das Programm lokal gestartet werden, initialisiert bei Bedarf die SQLite-Datenbank und stellt dem Benutzer ein menügesteuertes Interface zur Verfügung. Für die Nutzung ist keine zusätzliche Serverinfrastruktur erforderlich, was den Einstieg in Test- und Ausbildungsumgebungen erleichtert.

Für die Projektdokumentation empfiehlt sich ergänzend eine kurze Benutzerbeschreibung, in der die wichtigsten Menüpunkte, Eingabeanforderungen und typische Nutzungsschritte erläutert werden. Auch eine technische Kurzdokumentation zum Projektaufbau, zur Maven-Struktur und zur Ablage der SQLite-Datei ist zweckmäßig.

12. Fazit und Ausblick

Das Projekt zeigt, wie sich aus einer zunächst einfachen Konsolenanwendung schrittweise eine strukturierte, testbare und erweiterbare Fachanwendung entwickeln lässt. Besonders wertvoll war dabei die Überführung in eine klare Schichtenarchitektur, die Einführung einer SQLite-basierten Persistenz sowie die konsequente Absicherung zentraler Fachlogik durch automatisierte Tests.

Für spätere Ausbaustufen bieten sich mehrere Erweiterungen an, etwa vollständige CRUD-Funktionen für Datenbankobjekte, zusätzliche Repository-Tests, eine REST-Schnittstelle mit Spring Boot oder perspektivisch eine grafische Weboberfläche. Gerade diese Entwicklungsmöglichkeiten zeigen, dass die gewählte Architektur zukunftsfähig ist und sich nicht nur für das Ausbildungsprojekt, sondern auch als Lernbasis für anspruchsvollere Softwareprojekte eignet.